

DBMS FAST API CONNECTION

Step 1 — Upgrade pip tools

Run this in the VS Code terminal:

```
python -m pip install --upgrade pip setuptools wheel
```

Wait until it finishes.

Step 2 — Install a compatible greenlet version

Then run:

```
python -m pip install "greenlet==3.0.3"
```

Step 3 — Install SQLAlchemy

After greenlet succeeds:

```
python -m pip install sqlalchemy
```

Step 4 — Install PyMySQL

```
python -m pip install pymysql
```

Step 5 — Check everything

Run:

```
python -c "import fastapi, uvicorn, sqlalchemy, pymysql, greenlet; print('All installed successfully')"
```

OUTPUT:

```
PS C:\Users\anany\OneDrive\Desktop\FastAPI_University> python -m pip install --upgrade pip setuptools wheel
Requirement already satisfied: pip in c:\users\anany\appdata\local\programs\python\python311\lib\site-packages (24.0)
Collecting pip
  Downloading pip-26.2.1-py3-none-any.whl.metadata (4.6 kB)
Requirement already satisfied: setuptools in c:\users\anany\appdata\local\programs\python\python311\lib\site-packages (78.1.0)
Collecting setuptools
  Using cached setuptools-84.0.0-py3-none-any.whl.metadata (6.6 kB)
Collecting wheel
  Downloading wheel-0.48.0-py3-none-any.whl.metadata (2.3 kB)
Requirement already satisfied: packaging>=24.0 in c:\users\anany\appdata\local\programs\python\python311\lib\site-packages (from wheel) (26.3)
Downloading pip-26.2.1-py3-none-any.whl (1.8 MB)
1.8/1.8 MB 2.9 MB/s eta 0:00:00
Using cached setuptools-84.0.0-py3-none-any.whl (818 kB)
Downloading wheel-0.48.0-py3-none-any.whl (33 kB)
Installing collected packages: wheel, setuptools, pip
  Attempting uninstall: setuptools
    Found existing installation: setuptools 78.1.0
    Uninstalling setuptools-78.1.0:
```

```
Uninstalling setuptools-78.1.0:
  Successfully uninstalled setuptools-78.1.0
  Attempting uninstall: pip
    Found existing installation: pip 24.0
    Uninstalling pip-24.0:
      Successfully uninstalled pip-24.0
Successfully installed pip-26.2.1 setuptools-84.0.0 wheel-0.48.0
PS C:\Users\anany\OneDrive\Desktop\FastAPI_University> python -m pip install "greenlet==3.0.3"
Collecting greenlet==3.0.3
  Downloading greenlet-3.0.3-cp311-cp311-win_amd64.whl.metadata (3.9 kB)
Downloading greenlet-3.0.3-cp311-cp311-win_amd64.whl (292 kB)
Installing collected packages: greenlet
Successfully installed greenlet-3.0.3
PS C:\Users\anany\OneDrive\Desktop\FastAPI_University> python -m pip install sqlalchemy
Collecting sqlalchemy
  Downloading sqlalchemy-2.0.52-cp311-cp311-win_amd64.whl.metadata (9.9 kB)
Requirement already satisfied: greenlet>=1 in C:\Users\anany\AppData\Local\Programs\Python\Python311\Lib\site-packages (from sqlalchemy) (3.0.3)
Requirement already satisfied: typing-extensions>=4.6.0 in C:\Users\anany\AppData\Local\Programs\Python\Python311\Lib\site-packages (from sqlalchemy) (4.16.0)
```

```

Collecting sqlalchemy
  Downloading sqlalchemy-2.0.52-cp311-cp311-win_amd64.whl.metadata (9.9 kB)
Requirement already satisfied: greenlet>=1 in C:\Users\anany\AppData\Local\Programs\Python\Python311\Lib\site-packages (from sqlalchemy) (3.0.3)
Requirement already satisfied: typing-extensions>=4.6.0 in C:\Users\anany\AppData\Local\Programs\Python\Python311\Lib\site-packages (from sqlalchemy) (4.16.0)
Downloading sqlalchemy-2.0.52-cp311-cp311-win_amd64.whl (2.2 MB)
  2.2/2.2 MB 1.4 MB/s 0:00:01
Installing collected packages: sqlalchemy
Successfully installed sqlalchemy-2.0.52
PS C:\Users\anany\OneDrive\Desktop\FastAPI_University> python -m pip install pymysql
Collecting pymysql
  Downloading pymysql-1.2.0-py3-none-any.whl.metadata (4.3 kB)
Downloading pymysql-1.2.0-py3-none-any.whl (45 kB)
Installing collected packages: pymysql
Successfully installed pymysql-1.2.0
PS C:\Users\anany\OneDrive\Desktop\FastAPI_University> python -c "import fastapi, uvicorn, sqlalchemy, pymysql, greenlet; print('All installed successfully')"
All installed successfully
PS C:\Users\anany\OneDrive\Desktop\FastAPI_University>

```

Step 6-Create a database in MySQL

In MySQL Workbench:

```
CREATE DATABASE university;
```

Then:

```
USE university;
```

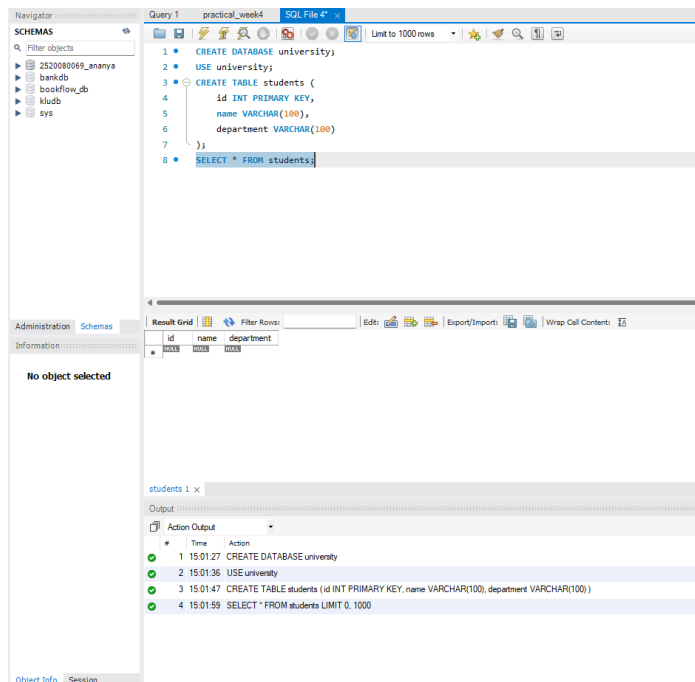
Create a student table:

```
CREATE TABLE students (
    id INT PRIMARY KEY,
    name VARCHAR(100),
    department VARCHAR(100)
);
```

Check it:

```
SELECT * FROM students;
```

OUTPUT:



Step 7 Now create a GET API

Let's retrieve students from MySQL.

from fastapi import FastAPI

from sqlalchemy import create_engine, text

```
app = FastAPI()
```

```
DATABASE_URL = "mysql+pymysql://root:your_password@localhost/university"
```

```
engine = create_engine(DATABASE_URL)
```

```
@app.get("/students")
```

```
def get_students():
```

```
with engine.connect() as connection:
```

```
    result = connection.execute(  
        text("SELECT * FROM students")  
    )
```

```
    students = result.mappings().all()
```

```
    return students
```

Now start FastAPI:

```
python -m uvicorn main:app --reload
```

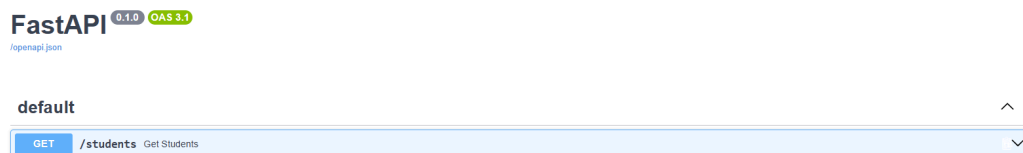
Open:

<http://127.0.0.1:8000/docs>

You'll see:

GET /students

OUTPUT:



Try it out → Execute

If your MySQL table contains:

1 | Isha | AI DS

2 | Rahul | CSE

FastAPI returns something like:

[

```

    {
      "id": 1,
      "name": "Isha",
      "department": "AI DS"
    },
    {
      "id": 2,
      "name": "Rahul",
      "department": "CSE"
    }
  ]

```

OUTPUT:

The screenshot shows a database management interface with a SQL editor and a results pane. The SQL editor contains the following code:

```

1 CREATE DATABASE university;
2 USE university;
3 CREATE TABLE students (
4   id INT PRIMARY KEY,
5   name VARCHAR(100),
6   department VARCHAR(100)
7 );
8 SELECT * FROM students;
9 INSERT INTO students (id, name, department)
10 VALUES
11 (1, 'Isha', 'AI & DS'),
12 (2, 'Rahul', 'CSE');
13

```

The results pane displays a table with the following data:

id	name	department
1	Isha	AI & DS
2	Rahul	CSE

The output pane shows the execution log:

#	Time	Action
1	15:01:27	CREATE DATABASE university
2	15:01:36	USE university
3	15:01:47	CREATE TABLE students (id INT PRIMARY KEY, name VARCHAR(100), department VARCHAR(100))
4	15:01:59	SELECT * FROM students LIMIT 0, 1000
5	15:08:57	INSERT INTO students (id, name, department) VALUES (1, 'Isha', 'AI & DS'), (2, 'Rahul', 'CSE')
6	15:09:04	SELECT * FROM students LIMIT 0, 1000

Responses

Curl

```
curl -X 'GET' \
'http://127.0.0.1:8000/students' \
-H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/students
```

Server response

Code	Details
200	Response body <pre>[{ "id": 1, "name": "Isha", "department": "AI & DS" }, { "id": 2, "name": "Rahul", "department": "CSE" }]</pre>  Download

Response headers

```
content-length: 90
content-type: application/json
date: Wed, 09 Sep 2026 09:42:38 GMT
server: uvicorn
```

Responses

Code	Description	Links
200	Successful Response Media type application/json Controls Accept header. Example Value Schema <pre>"string"</pre>	No links

Step 8. Now POST can actually save data

from pydantic import BaseModel

class Student(BaseModel):

id: int

name: str

department: str

Then:

@app.post("/students")

def create_student(student: Student):

with engine.begin() as connection:

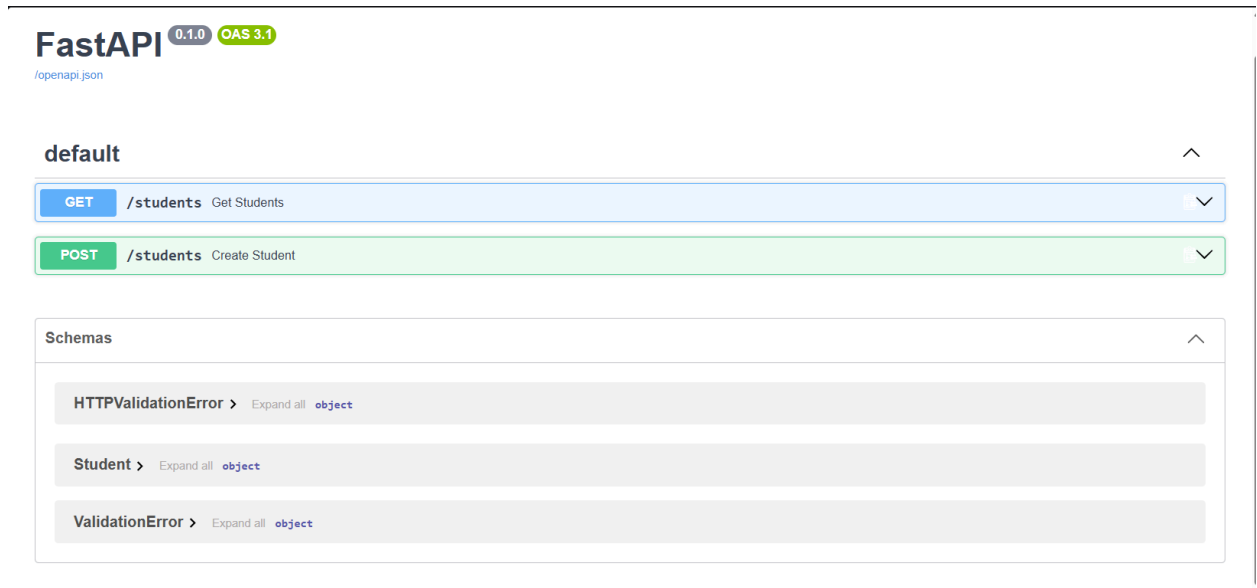
connection.execute(

```

text("""
INSERT INTO students (id, name, department)
VALUES (:id, :name, :department)
"""),
{
    "id": student.id,
    "name": student.name,
    "department": student.department
}
)
return {
    "message": "Student created",
    "student": student
}

```

OUTPUT:



The screenshot displays the FastAPI Swagger UI interface. At the top, it shows the FastAPI logo with version 0.1.0 and OAS 3.1. Below the logo, the 'default' API is listed. Two endpoints are visible: a GET endpoint for '/students' (Get Students) and a POST endpoint for '/students' (Create Student). The 'Schemas' section is expanded, showing three schemas: HTTPValidationError, Student, and ValidationError. Each schema has an 'Expand all' button and a link to the 'object' definition.

Now send through Swagger:

```

{
    "id": 3,
    "name": "Ansh",

```

```
"department": "CSE"
}
```

OUTPUT:

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/students' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "id": 3,
    "name": "Ansh",
    "department": "CSE"
  }'
```

Request URL

http://127.0.0.1:8000/students

Server response

Code Details

200

Response body

```
{
  "message": "Student created",
  "student": {
    "id": 3,
    "name": "Ansh",
    "department": "CSE"
  }
}
```

Response headers

```
content-length: 81
content-type: application/json
date: Wed, 09 Sep 2026 09:53:19 GMT
```

This POST change is reflected in the MySQL Workbench

Navigator

Query 1 practical_week4 SQL File 4

Limit to 1000 rows

1 CREATE DATABASE university;

2 USE university;

3 CREATE TABLE students (

4 id INT PRIMARY KEY,

5 name VARCHAR(100),

6 department VARCHAR(100)

7);

8 SELECT * FROM students;

9 INSERT INTO students (id, name, department)

10 VALUES

11 (1, 'Isha', 'AI & DS'),

12 (2, 'Rahul', 'CSE');

13

Administration Schemas

Information

No object selected

Result Grid

	id	name	department
1	1	Isha	AI & DS
2	2	Rahul	CSE
3	3	Ansh	CSE

students 3 x

Output

Action Output

#	Time	Action	Message
1	15:01:27	CREATE DATABASE university	1 row(s) affected
2	15:01:36	USE university	0 row(s) affected
3	15:01:47	CREATE TABLE students (id INT PRIMARY KEY, name VARCHAR(100), department VARCHAR(100))	0 row(s) affected
4	15:01:59	SELECT * FROM students LIMIT 0, 1000	0 row(s) returned
5	15:08:57	INSERT INTO students (id, name, department) VALUES (1, 'Isha', 'AI & DS'), (2, 'Rahul', 'CSE')	2 row(s) affected Records: 2 Duplicates: 0 Warnings: 0
6	15:09:04	SELECT * FROM students LIMIT 0, 1000	2 row(s) returned
7	15:26:27	SELECT * FROM students LIMIT 0, 1000	3 row(s) returned

Object Info Session

Step 9 : Do the same for POST

```
@app.put("/students/{student_id}")
```

```
def update_student(student_id: int, student: Student):
```

```
    with engine.begin() as connection:
```

```
        connection.execute(
```

```
            text("""
```

```
                UPDATE students
```

```
                SET id = :new_id,
```

```
                    name = :name,
```

```
                    department = :department
```

```
                WHERE id = :old_id
```

```
            """),
```

```
            {
```

```
                "old_id": student_id,
```

```
                "new_id": student.id,
```

```
                "name": student.name,
```

```
                "department": student.department
```

```
            }
```

```
        )
```

```
    return {
```

```
        "message": "Student updated",
```

```
        "student": student
```

```
    }
```

OUTPUT:

FastAPI 0.1.0 **OAS 3.1**
/openapi.json

default ^

- GET** /students Get Students
- POST** /students Create Student
- PUT** /students/{student_id} Update Student

Schemas ^

- HTTPValidationError** > Expand all object
- Student** > Expand all object
- ValidationError** > Expand all object

POST /students Create Student

PUT /students/{student_id} Update Student ^

Parameters Cancel Reset

Name	Description
student_id * required	
integer	
(path)	3

Request body required application/json

Edit Value | Schema

```
{
  "id": 3,
  "name": "Arsh",
  "department": "CSE"
}
```

Execute Clear

Responses

responses

Curl

```
curl -X 'PUT' \
  'http://127.0.0.1:8000/students/3' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "id": 3,
    "name": "Ansh",
    "department": "CSE"
  }'
```

Request URL

http://127.0.0.1:8000/students/3

Server response

Code	Details
200	Response body <pre>{ "message": "Student updated", "student": { "id": 3, "name": "Ansh", "department": "CSE" } }</pre> Response headers <pre>content-length: 81 content-type: application/json date: Thu, 10 Sep 2026 13:31:16 GMT server: uvicorn</pre>

Responses

Code	Description	Links
200	Successful Response Media type application/json Controls Accept header. Example Value Schema	No links

Step 10 : Do the same for DELETE

@app.delete("/students/{student_id}")

def delete_student(student_id: int):

with engine.begin() as connection:

connection.execute(

text("""

DELETE FROM students

WHERE id = :id

"""),

{

"id": student_id

}

)

return {

"message": "Student deleted"

}

OUTPUT:

FastAPI 0.1.0 **QAS 3.1**
/openapi.json

default ^

- GET** /students Get Students
- POST** /students Create Student
- PUT** /students/{student_id} Update Student
- DELETE** /students/{student_id} Delete Student

Schemas ^

- HTTPValidationError** > Expand all object
- Student** > Expand all object
- ValidationError** > Expand all object

Parameters Cancel

Name	Description
student_id * required Integer (path)	2

Execute Clear

Responses

Curl

```
curl -X 'DELETE' \
'http://127.0.0.1:8000/students/2' \
-H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/students/2
```

Server response

Code	Details
200	Response body <pre>{ "message": "Student deleted" }</pre> Download Response headers <pre>content-length: 29 content-type: application/json date: Thu, 10 Sep 2026 14:31:53 GMT server: uvicorn</pre>

Responses